# Aura Programming Language: User Manual

Welcome to the Aura programming language!

Aura is a simple educational programming language designed to help students learn compiler construction. This version focuses on variable declaration, arithmetic operations, input/output, and code documentation with unique comment symbols.

**1. Getting Started**

- File extension: .aura
- Running a program: Compile and run using the current Aura compiler workflow:

# Build the compiler

***bison -d -o parser/aura_parser.tab.c parser/aura_parser.y***
***flex -o lexer/lex.yy.c lexer/aura_lexer.l***
***gcc -Wall -g -o aura parser/aura_parser.tab.c lexer/lex.yy.c -lfl***

# Run a program

***./aura examples/test.aura***

- Output: The compiler prints assigned values, user inputs, and evaluation results, followed by a symbol table.

**2. Basic Syntax**

- Case-sensitive: Name and name are different variables.
- Whitespace: Ignored except inside strings.
- Statement terminator: Each statement must end with a semicolon (;).
- Reserved keywords: Cannot be used as variable names.

Current reserved keywords:

***var, number, decimal, text, input, output***

**Comments:**

- Single-line: Start with ~# and continue to the end of the line.
- Multi-line: Start with ~* ***and end with*** *~

### 3. Variables

Variables are declared with the var keyword, followed by a type:

| Type | Description |
|---|---|
| number | Integer numbers |
| decimal | Floating-point numbers |
| text | Strings |

Declaration examples:

```
~# Declaration with initialization
var number age = 21;
var decimal pi = 3.1416;
var text name = "Alice";

~# Declaration without initialization
var number count;
var text title;

Assignment (after declaration):
age = age + 1;
name = "Bob";
```

### 4. Input and Output

Input

Read user input using:
input variable;

- Input type must match variable type (number, decimal, text).

*Example:*

```
var text username;
output "Enter your name:";
input username;
```

*Output*

*Print one or more expressions separated by commas:*

```
output expr1, expr2, ...;
```

- Expressions can be literals or variables.

*Example:*

```
output "Hello,", username, "!";
var number age = 21;
output "Next year you will be", age + 1;
```

5. Comments

*Single-line:*

```
~# This is a single-line comment
```

*Multi-line:*

```
~*
This is a
multi-line comment
*~
```

Comments can appear anywhere in the program.

## 6. Expressions

Aura supports simple arithmetic expressions:

- Operators: +, -, *, /
- Operands: integer variables, decimal variables, integer literals, decimal literals
- Parentheses allowed for grouping.

*Examples:*

```
var number a = 5;
var decimal b = 2.5;
var number c;
c = a + 10;
var decimal d;
d = b * 3.0;
output c, d;
```

- Division by zero triggers a runtime error.

## 7. Example Program

This program demonstrates variable declaration, input/output, arithmetic, and comments:

```
~# Welcome program
var text name = "Alice";
output "Name:";
output name;

~# Working with numbers
var number age = 21;
age = age + 1;
output "Next age:";
output age;

~# User input example
input name;
output "Hello,", name;
```

**Expected Output:**

```
$ ./aura examples/test.aura
=== Aura Language Compiler ===
Starting compilation...

Assigned name = "Alice"
Name:
Alice
Assigned age = 21
Assigned age = 22
Next age:
22
Mohini
Hello, Mohini
```

### 8. Notes and Restrictions

- ❖ Semicolons (;) are required at the end of statements.
- ❖ Only comma-separated expressions are allowed in output statements.
- ❖ Variable names must not match reserved keywords.
- ❖ % operator is not supported.
- ❖ String literals are automatically stripped of quotes when stored and printed.

### 9. Symbol Table

After compilation, the compiler prints the symbol table showing all declared variables:

```
=== Compilation Complete ===
Symbol Table:
  name (text)
  age (number)
```

### 10. Conclusion

Aura is a beginner-friendly, educational programming language to:

- Declare and use variables (number, decimal, text)
- Perform simple arithmetic operations
- Use input/output operations with user interaction
- Document code with unique single- and multi-line comments

This version is ideal for learning compiler construction, lexical analysis, and parsing in a clear, structured way.